



Hands-On Lab Exercise 2: Simple Task Automation Using Shell Script

Topic: Learning Shell Script Through Real System Tasks

Author: Dr. Sandeep Kumar Sharma

This lab focuses on **simple real-life automation examples** using shell scripts. No variables, no conditions, no loops — only simple commands to understand how shell scripts work.

The goal is to understand:

How a shell script performs tasks automatically step-by-step.



Learning Objective

- Understand how shell scripts automate tasks
 - Learn to group multiple commands in one script
 - Understand script execution flow
 - Learn real system task automation
 - Build confidence in writing task-based scripts
-



Learning Outcome

After this lab, participants will: - Create task-based scripts - Automate directory and file creation - Automate package installation - Collect system information - Use shell scripts for daily system operations



Example 1: Machine Setup Script (Directory + Files + Messages)



Purpose:

This script prepares a basic project structure automatically.

Step 1: Create Script File

```
touch setup.sh
```

Step 2: Open File

```
nano setup.sh
```

Step 3: Write Script

```
#!/bin/bash

echo "Starting system setup..."

echo "Creating project directories..."
mkdir project
mkdir project/app
mkdir project/logs
mkdir project/config

echo "Creating files..."
touch project/app/app.txt
touch project/logs/log.txt
touch project/config/config.txt

echo "Setup completed successfully"
```

Step 4: Give Permission

```
chmod +x setup.sh
```

Step 5: Run Script

```
./setup.sh
```

What This Script Does:

- Creates folders
 - Creates files
 - Shows messages
 - Automates full setup in one run
-

Example 2: Package Installation Script

Purpose:

This script installs basic packages automatically.

Step 1: Create Script

```
touch install.sh
```

Step 2: Open File

```
nano install.sh
```

Step 3: Write Script

For Ubuntu/Debian:

```
#!/bin/bash

echo "Updating system..."
apt update

echo "Installing packages..."
apt install -y git
apt install -y curl
apt install -y wget
```

```
echo "Installation completed"
```

For RHEL/CentOS:

```
#!/bin/bash

echo "Updating system..."
yum update -y

echo "Installing packages..."
yum install -y git
yum install -y curl
yum install -y wget

echo "Installation completed"
```

Step 4: Permission

```
chmod +x install.sh
```

Step 5: Run

```
./install.sh
```



Example 3: System Information Script



Purpose:

This script shows system and user information.

Step 1: Create Script

```
touch system_info.sh
```

Step 2: Open File

```
nano system_info.sh
```

Step 3: Write Script

```
#!/bin/bash

echo "System Information"

echo "-----"

echo "Current Date and Time:"
date

echo "-----"

echo "Current User:"
whoami

echo "-----"

echo "Present Working Directory:"
pwd

echo "-----"

echo "System Uptime:"
uptime

echo "-----"

echo "Disk Usage:"
df -h

echo "-----"
```

```
echo "Memory Usage:"  
free -h
```

Step 4: Permission

```
chmod +x system_info.sh
```

Step 5: Run

```
./system_info.sh
```

What This Script Shows:

- Date and time
- Logged-in user
- Current directory
- System uptime
- Disk usage
- Memory usage

Understanding the Concept

These scripts show that: - Shell scripts are task automation tools - Multiple commands can run together - Scripts follow top-to-bottom execution - Scripts reduce manual work - Scripts make system operations easy

Summary

- Shell scripts automate daily tasks
- One script can perform multiple actions
- Automation increases efficiency
- Scripts are reusable
- Scripts are reliable

End of Lab

This lab shows how shell scripts work in real tasks like: - Machine setup - Software installation - System monitoring

These are the building blocks of automation systems.

Author:

Dr. Sandeep Kumar Sharma



LinkedIn:

linkedin.com/in/sandeep-kaushik-2a345856

Medium Blog:

medium.com/@shyamsandeep28